

编译原理lab2实验报告

一、实验完成情况

主要完成以下语法分析功能：

1. 在无词法和语法错误时，执行语义分析入口并遍历整棵语法树。
2. 建立变量、函数、结构体三类符号表。
3. 实现基础类型、数组类型、结构体类型、函数类型和错误类型的统一类型系统。
4. 实现错误类型 1 到 17 的语义检查与规范化输出，无错误时不输出。
5. 选做3.2：变量采用可嵌套作用域规则。

二、功能实现

2.1 总体流程

语义分析流程如下：

1. 解析输入文件，得到语法树。
2. 若存在词法或语法错误，则跳过语义分析。
3. 对 Program 节点执行语义遍历。
4. 在遍历过程中完成符号登记、表达式类型计算与错误检测。
5. 按 Error type k at Line n: message. 统一格式输出错误。

实现方式：

1. parse_file 负责状态重置、调用 yyparse 和语义入口 analyze_program。
2. analyze_program -> analyze_extdeflist -> analyze_extdef 构成顶层定义遍历链。
3. 表达式由 analyze_exp 递归分析并返回 ExprInfo（类型 + 是否左值）。
4. 类型判等由 same_type 统一处理。

2.2 符号与类型系统

实现了三套符号信息：

1. 变量符号：名称、类型、行号。
2. 函数符号：名称、函数签名、行号。

3. 结构体符号：名称、字段列表、字段数量、行号。

实现了统一 Type 结构：

1. BASIC：int 或 float。
2. ARRAY：元素类型 + 长度。
3. STRUCT：结构体名。
4. FUNCTION：返回类型 + 形参类型序列。
5. ERROR：错误传播类型，避免级联误报。

2.3 作用域机制

作用域通过作用域栈实现：

1. enter_scope 入栈，记录当前变量数量作为作用域基线。
2. leave_scope 出栈，恢复变量数量，等价于销毁内层局部变量。
3. add_var 仅在当前作用域范围内检查重名，从而允许外层与内层同名遮蔽。
4. 不同函数体内局部变量自然可重名（函数体各自入栈）。

2.4 Error type 1-17

- 类型 1：使用未定义变量。analyze_exp 中 ID 分支，变量和函数都查不到则报错。
- 类型 2：调用未定义函数。函数调用分支中函数表查找失败报错。
- 类型 3：变量重定义，或变量名与已有结构体名冲突。add_var 中处理。
- 类型 4：函数重定义。define_function 中处理。
- 类型 5：赋值两侧类型不匹配。赋值表达式和声明初始化中处理。
- 类型 6：赋值左侧不是左值。赋值表达式中通过 is_lvalue 判断。
- 类型 7：操作数与操作符不匹配。算术、逻辑、关系、if/while 条件统一校验。
- 类型 8：return 类型与函数返回类型不匹配。analyze_stmt 中 RETURN 分支处理。
- 类型 9：函数实参与形参数目或类型不匹配。调用分支中检查。
- 类型 10：对非数组进行下标访问。Exp LB Exp RB 分支处理。
- 类型 11：对普通变量使用函数调用。函数调用分支中变量表命中时处理。
- 类型 12：数组下标不是整数。下标表达式类型检查处理。
- 类型 13：对非结构体使用点操作符。Exp DOT ID 分支处理。
- 类型 14：访问结构体中不存在的字段。字段查找失败时报错。
- 类型 15：结构体字段重复，或字段定义时初始化。结构体定义分析中处理。
- 类型 16：结构体名与已有结构体或变量名冲突。add_struct_symbol 中处理。
- 类型 17：使用未定义结构体定义变量。结构体引用分支处理。

补充：

1. 错误去重：同一行、同一错误类型只报一次。
2. 错误类型传播：使用 TYPE_ERROR 缓冲后续推导。
3. 数组类型等价：忽略具体长度，只比较基类型与维度。
4. 结构体类型等价：使用名等价（结构名比较）；匿名结构体通过内部隐藏名区分。

三、编译运行和测试

1. cd Lab/Code
2. make clean
3. make
4. ./parser ../Test/test{1-29}.cmm，其中1-23为实验手册所给出的test，24-29为自测样例，关注复杂代码的错误是否报全（参考lab1）。